

Name: Y.Veerendar Reddy

Reg.No:- 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT - 3

Policy and Value Function Representation Using Bellman Equations

Aim

To implement **policy and value functions** using the **Bellman equation** and analyze the value of different states in a simple Reinforcement Learning environment.

Procedure

1. Define the states and possible actions.
2. Define the rewards associated with state transitions.
3. Define a fixed policy for selecting actions.
4. Initialize the value of each state.
5. Apply the Bellman equation repeatedly.
6. Update the value of each state until the values become stable.
7. Display the calculated state values.
8. Visualize the value function for all states.

Code:

```
# =====  
# EXPERIMENT 3  
# Policy and Value Function Representation  
# Using Bellman Equations  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
# -----  
# STEP 1: Define States  
# -----
```

```

states = ["Start", "Middle", "Goal"]

# Number of states
num_states = len(states)

# -----
# STEP 2: Define Actions
# -----

actions = ["Move", "Stay"]

# -----
# STEP 3: Define Discount Factor
# -----

gamma = 0.9

# -----
# STEP 4: Define Policy
# -----
# The policy tells the agent which action to choose
# in each state.

policy = {
    "Start": "Move",
    "Middle": "Move",
    "Goal": "Stay"
}

# -----

```

```
# STEP 5: Define Rewards
```

```
# -----
```

```
rewards = {  
    ("Start", "Move"): 1,  
    ("Start", "Stay"): -1,  
  
    ("Middle", "Move"): 10,  
    ("Middle", "Stay"): -1,  
  
    ("Goal", "Move"): 0,  
    ("Goal", "Stay"): 0  
}
```

```
# -----
```

```
# STEP 6: Define State Transitions
```

```
# -----
```

```
def transition(state, action):
```

```
    if state == "Start":
```

```
        if action == "Move":
```

```
            return "Middle"
```

```
        else:
```

```
            return "Start"
```

```
    elif state == "Middle":
```

```
        if action == "Move":
```

```
            return "Goal"
```

```

        else:
            return "Middle"

    elif state == "Goal":

        return "Goal"

# -----
# STEP 7: Initialize Value Function
# -----

value_function = {
    "Start": 0.0,
    "Middle": 0.0,
    "Goal": 0.0
}

# -----
# STEP 8: Bellman Equation Iteration
# -----

iterations = 20

value_history = {
    "Start": [],
    "Middle": [],
    "Goal": []
}

for iteration in range(iterations):

```

```

new_values = {}

for state in states:

    # Select action according to policy
    action = policy[state]

    # Obtain immediate reward
    reward = rewards[(state, action)]

    # Find next state
    next_state = transition(state, action)

    # Bellman equation
    new_value = reward + gamma * value_function[next_state]

    new_values[state] = new_value

# Update value function
value_function = new_values

# Store values for visualization
for state in states:
    value_history[state].append(value_function[state])

# -----
# STEP 9: Display Final Value Function
# -----

```

```

print("=" * 55)
print("POLICY AND VALUE FUNCTION")
print("=" * 55)

print("\nPolicy:")

for state in states:
    print(f"{state} -> {policy[state]}")

print("\nFinal Value Function:")

for state in states:
    print(f"V({state}) = {value_function[state]:.4f}")

# -----
# STEP 10: Display Bellman Calculation
# -----

print("\n" + "=" * 55)
print("BELLMAN EQUATION CALCULATION")
print("=" * 55)

for state in states:

    action = policy[state]
    reward = rewards[(state, action)]
    next_state = transition(state, action)

    print(
        f"V({state}) = {reward} + "

```

```
        f"{gamma} * V({next_state})"
    )
```

```
# -----
# STEP 11: Visualization of Final Value Function
# -----
```

```
plt.figure(figsize=(8, 5))
```

```
values = [value_function[state] for state in states]
```

```
plt.bar(states, values)
```

```
plt.title("State Value Function")
```

```
plt.xlabel("States")
```

```
plt.ylabel("Value")
```

```
plt.grid(axis="y")
```

```
plt.show()
```

```
# -----
# STEP 12: Visualization of Value Convergence
# -----
```

```
plt.figure(figsize=(10, 6))
```

```
for state in states:
```

```
plt.plot(  
    range(1, iterations + 1),  
    value_history[state],  
    marker="o",  
    label=state  
)
```

```
plt.title("Value Function Convergence")  
plt.xlabel("Iteration")  
plt.ylabel("State Value")
```

```
plt.legend()  
plt.grid(True)
```

```
plt.show()
```

OUTPUT:-

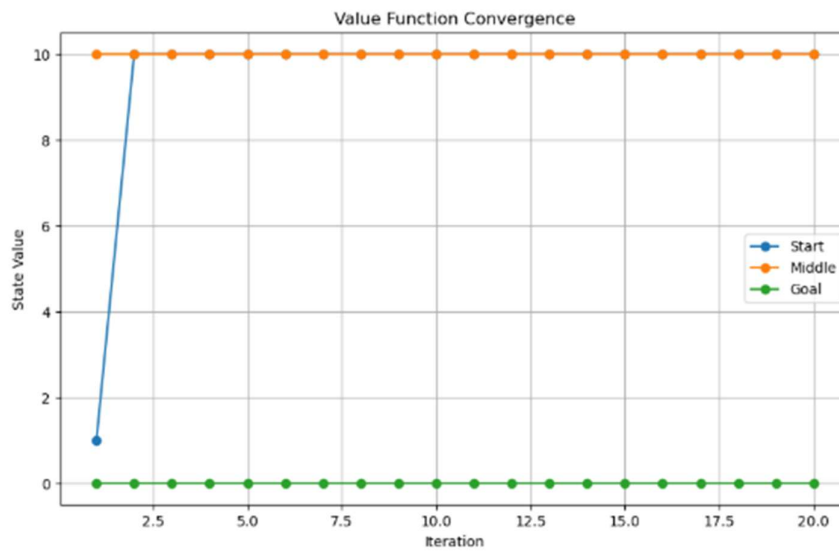
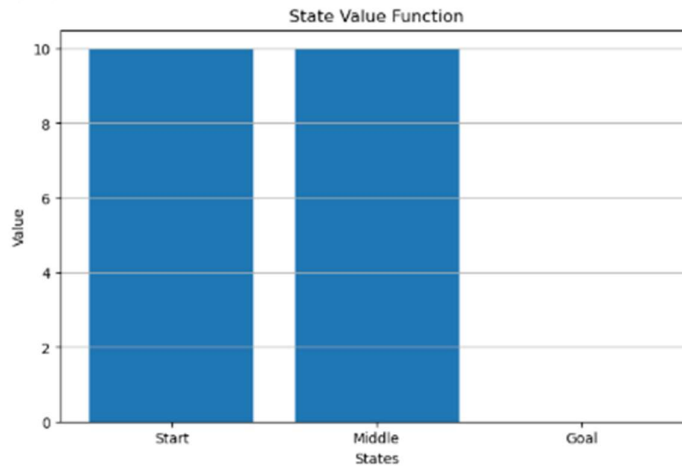
POLICY AND VALUE FUNCTION

Policy:
Start -> Move
Middle -> Move
Goal -> Stay

Final Value Function:
 $V(\text{Start}) = 10.0000$
 $V(\text{Middle}) = 10.0000$
 $V(\text{Goal}) = 0.0000$

BELLMAN EQUATION CALCULATION

$V(\text{Start}) = 1 + 0.9 * V(\text{Middle})$
 $V(\text{Middle}) = 10 + 0.9 * V(\text{Goal})$
 $V(\text{Goal}) = 0 + 0.9 * V(\text{Goal})$



Visualization

The program produces two visualizations:

1. State Value Function

A bar graph represents the final value of:

- Start
- Middle
- Goal

The state with a higher value represents a more desirable state under the selected policy.

2. Value Function Convergence

A line graph shows how the values of the states change during successive Bellman equation iterations.

As the number of iterations increases, the state values approach their stable values.

Summary

The policy and value functions were implemented using the Bellman equation. The value of each state was repeatedly updated using the immediate reward and discounted value of the next state.

Result

Thus, the policy and value functions were successfully represented using the Bellman equation, and the convergence of state values was visualized.